Universidade Federal de São Carlos

Departamento de Computação

Arquitetura e Organização de Computadores 2

Relatório de Atividades Práticas - Parte 2

Integrantes:

Cauã Borges Faria – RA: 834437
Murilo Oliva Mantovani – RA: 834730
Nicolas Magno Messias da Silva – RA: 834054

31 de outubro de 2025

Conteúdo

1 Relatório Semana 10 **2**
1.1 Especificação e Objetivos do Problema . . . 2
1.2 Metodologia . . . 2
1.3 Desenvolvimento e Experimentos Práticos . . . 3
1.3.1 Lição 5: Controle de LED com Botão (Sinais Digitais) . . . 3
1.3.2 Lição 6: Controle de LED com Potenciômetro (Sinais Analógicos) . . . 5
1.3.3 Resumo: Classe Pin da Biblioteca machine . . . 7
1.4 Conclusões . . . 10
1.4.1 Aspectos Teóricos Relevantes: . . . 10
1.4.2 Aspectos Práticos Relevantes: . . . 11
1.4.3 Comparação Prática: . . . 11
1.4.4 Experiência do Grupo: . . . 11
1.5 Vídeo de Apresentação . . . 12

2 Relatório Semana 11 **13**
2.1 Especificação e Objetivos do Problema . . . 13
2.2 Fundamentação Teórica . . . 13
2.3 Lição 7: Controle de LED com PWM . . . 13
2.4 Lição 8: Controle de Buzzer com PWM . . . 15
2.5 Resumo das Classes da Biblioteca `machine` . . . 16
2.6 Conclusões . . . 17
2.7 Vídeo de Apresentação . . . 17

3 Relatório Semana 12 **18**
3.1 Especificação e Objetivos do Problema . . . 18
3.2 Metodologia . . . 18
3.3 Desenvolvimento e Experimentos Práticos . . . 19
3.3.1 Fundamentação Teórica: LCD e Cristal Líquido . . . 19
3.3.2 Barramento I^2C no Raspberry Pi Pico . . . 20
3.3.3 Implementação da Lição 9: Exibição de Mensagens . . . 20
3.3.4 Resumo: Biblioteca Externa `lcd1602.py` . . . 21
3.4 Conclusões . . . 22
3.4.1 Experiência do Grupo . . . 23
3.5 Vídeo de Apresentação . . . 23

4 Relatório Semana 13 – Projeto Final **24**
4.1 Especificação e Objetivos do Projeto . . . 24
4.2 Metodologia . . . 24
4.3 Desenvolvimento e Experimentos Práticos . . . 25
4.3.1 Fundamentação Teórica . . . 25
4.3.2 Código Implementado . . . 26
4.3.3 Resultados Observados . . . 28
4.4 Conclusões . . . 29
4.5 Vídeo de Apresentação . . . 29

1 Relatório Semana 10

1.1 Especificação e Objetivos do Problema

Esta atividade prática teve como objetivo compreender e aplicar conceitos fundamentais relacionados ao tratamento de sinais digitais e analógicos em microcontroladores, especificamente no Raspberry Pi PICO. Os objetivos específicos foram:

1. **Lição 5 – Sinais Digitais:**
   - Compreender o conceito de sinais digitais (binários: 0 e 1)
   - Implementar circuito com botão para controle de LED
   - Praticar leitura de entrada digital e controle de saída digital
2. **Lição 6 – Sinais Analógicos:**
   - Entender a natureza contínua dos sinais analógicos
   - Conhecer o funcionamento do conversor ADC (Analog to Digital Converter)
   - Utilizar sensor de rotação (potenciômetro) para controlar LED
   - Trabalhar com as portas ADC do Pico (GP26, GP27, GP28 / A0, A1, A2)
3. **Documentação:**
   - Elaborar resumo sobre funcionalidades da classe `Pin` da biblioteca `machine`
   - Estudar a documentação oficial do MicroPython

1.2 Metodologia

Para realizar os experimentos, utilizamos:

Hardware:

- Raspberry Pi PICO com microcontrolador RP2040
- Shield for Pi Pico (para facilitar conexões)
- Botão (pushbutton) para entrada digital
- LED para saída visual
- Sensor de rotação (potenciômetro) para entrada analógica

Software:

- IDE Thonny configurada para Raspberry Pi PICO
- MicroPython como linguagem de programação
- Biblioteca `machine` para acesso aos pinos GPIO

Documentação Consultada:

- SEED: Beginner's Guide to Raspberry Pi PICO
- MicroPython Libraries – Repositório Oficial: `https://docs.micropython.org/en/latest/library/index.html`
- Fóruns da Raspberry Pi Foundation

1.3 Desenvolvimento e Experimentos Práticos

1.3.1 Lição 5: Controle de LED com Botão (Sinais Digitais)

Fundamentação Teórica: Um sinal digital representa informação na forma binária, possuindo apenas dois estados possíveis:

- **Estado 0 (LOW):** Aproximadamente 0V – circuito desligado
- **Estado 1 (HIGH):** Aproximadamente 3.3V no Pico – circuito ligado

Essa simplicidade binária é fundamental para o processamento de informações em microcontroladores, uma vez que o chip RP2040 é composto por transistores que operam alternando entre esses dois estados.

Implementação Prática: O circuito implementado consiste em:

- **Entrada:** Botão conectado a um pino GPIO configurado como entrada digital
- **Saída:** LED conectado ao pino GPIO 16, configurado como saída digital
- **Lógica:** Quando o botão é pressionado (sinal HIGH), o LED acende; quando solto (sinal LOW), o LED apaga

Listing 1: Controle de LED com Botão – Sinais Digitais

```
import machine
import utime

# Configuracao do LED como saida digital (pino 16)
LED = machine.Pin(16, machine.Pin.OUT)

# Configuracao do botao como entrada digital (pino 14)
# Pull-down interno ativado para evitar leituras flutuantes
botao = machine.Pin(18, machine.Pin.IN, machine.Pin.PULL_DOWN)

print("Sistema iniciado. Pressione o botao para controlar o LED.")

# Loop principal
while True:
    # Le o estado do botao
    estado_botao = botao.value()

    # Controla o LED baseado no estado do botao
    if estado_botao == 1:  # Botao pressionado
        LED.value(1)       # Liga LED
    else:                   # Botao solto
        LED.value(0)       # Desliga LED

    # Pequeno delay para estabilidade
    utime.sleep_ms(50)
```

Análise do Código:

1. **Configuração de Pinos:**
   - `machine.Pin(16, machine.Pin.OUT)`: Configura GPIO 16 como saída para o LED
   - `machine.Pin(14, machine.Pin.IN, machine.Pin.PULL_DOWN)`: Configura GPIO 14 como entrada com resistor pull-down interno
2. **Resistor Pull-down:**
   - Essencial para evitar leituras flutuantes quando o botão não está pressionado
   - Garante que o pino leia 0V (LOW) quando o botão está aberto
3. **Loop de Leitura:**
   - `botao.value()` retorna 0 ou 1, representando o estado digital atual
   - O LED responde imediatamente ao estado do botão

Resultados Observados:

- ✓ LED acende instantaneamente ao pressionar o botão
- ✓ LED apaga imediatamente ao soltar o botão
- ✓ Comportamento estável sem flutuações (graças ao pull-down)
- ✓ Tempo de resposta imperceptível ao usuário (∼50ms)

1.3.2 Lição 6: Controle de LED com Potenciômetro (Sinais Analógicos)

Fundamentação Teórica: Sinais analógicos representam informações através de variações contínuas em amplitude e frequência. Diferentemente dos sinais digitais, que possuem apenas dois estados discretos, sinais analógicos podem assumir infinitos valores dentro de uma faixa.

Por que o ADC é necessário? O microcontrolador RP2040 é fundamentalmente digital – seus transistores operam apenas com 0s e 1s. Para processar sinais analógicos do mundo real (temperatura, luz, som, posição), é necessário converter esses sinais contínuos em valores digitais discretos. Esse é o papel do **ADC (Analog to Digital Converter)**.

Especificações do ADC no Raspberry Pi PICO:

- **Resolução:** 12 bits (valores de 0 a 4095)
- **Tensão de referência:** 3.3V
- **Canais disponíveis:** 3 canais externos + 1 interno (temperatura)
  - GP26 → A0
  - GP27 → A1
  - GP28 → A2
- **Precisão:** Cada unidade representa aproximadamente 0.8mV (3.3V / 4096)

Implementação Prática: O circuito implementado consiste em:

- **Entrada:** Potenciômetro (sensor de rotação) conectado à porta A0 (GP26)
- **Processamento:** ADC converte a tensão variável (0-3.3V) em valor digital (0-4095)
- **Saída:** LED que acende quando o valor lido ultrapassa um limiar definido

Listing 2: Controle de LED com Potenciômetro – Sinais Analógicos

```
import machine
import utime

LED = machine.Pin(16, machine.Pin.OUT)

sensor_rotacao = machine.ADC(1)

LIMIAR = 30000

print("Sistema de monitoramento analogico iniciado.")
print("Gire o potenciometro para controlar o LED.")

while True:
    valor_lido = sensor_rotacao.read_u16()
    percentual = (valor_lido / 65535) * 100
    tensao = (valor_lido / 65535) * 3.3

    print(f"Valor: {valor_lido:5d} | {percentual:5.1f}% | {tensao:.2f}V", e

    if valor_lido > LIMIAR:
        LED.value(1)
        print(" | LED: LIGADO")
    else:
        LED.value(0)
        print(" | LED: DESLIGADO")

    utime.sleep_ms(200)
```

Análise do Código:

1. **Configuração do ADC:**
   - `machine.ADC(26)`: Cria objeto ADC no pino GP26 (porta A0 do Shield)
   - O ADC é automaticamente configurado para modo de leitura
2. **Leitura de Valores:**
   - `read_u16()`: Retorna valor de 16 bits (0 a 65535) para maior precisão
   - Alternativa: `read_u16()` oferece melhor resolução que leitura de 12 bits
3. **Conversões Realizadas:**
   - **Para percentual:** $(valor/65535) \times 100$
   - **Para tensão:** $(valor/65535) \times 3.3V$
   - Essas conversões facilitam a interpretação dos dados

4. **Lógica de Controle:**
   - Define limiar de 30000 (∼46% da escala)
   - LED acende quando potenciômetro ultrapassa esse valor
   - Sistema funciona como comparador digital de sinal analógico

Resultados Observados:

✓ Leitura suave e contínua do potenciômetro

✓ Valores variam de 0 a 65535 conforme rotação

✓ LED responde precisamente ao limiar definido

✓ Conversões para percentual e tensão auxiliam na calibração

✓ Taxa de atualização de 200ms proporciona resposta fluida

Tabela 1: Leituras do potenciômetro em diferentes posições

Posição	**Valor Digital**	**Percentual**	**Tensão**	**Estado LED**
Mínimo (0°)	∼100	0.2%	0.01V	Desligado
25%	∼16384	25.0%	0.83V	Desligado
50% (Limiar)	∼32768	50.0%	1.65V	Ligado
75%	∼49152	75.0%	2.48V	Ligado
Máximo (270°)	∼65435	99.8%	3.29V	Ligado

Tabela de Leituras Típicas:

1.3.3 Resumo: Classe Pin da Biblioteca machine

A classe `Pin` é fundamental para interação com os pinos GPIO do Raspberry Pi PICO, permitindo configurar e controlar entradas e saídas digitais.

Listing 3: Sintaxe básica da classe Pin

```
# Sintaxe basica
pino = machine.Pin(numero_pino, modo, pull)

# Exemplos:
led = machine.Pin(16, machine.Pin.OUT)                    # Saida
botao = machine.Pin(14, machine.Pin.IN)                   # Entrada
botao_pull = machine.Pin(14, machine.Pin.IN, machine.Pin.PULL_UP)
# Entrada com pull-up
```

Modos de Operação:

1. **machine.Pin.OUT** – Saída Digital
   - Permite enviar sinais HIGH (1) ou LOW (0)
   - Usado para LEDs, relés, acionamentos
2. **machine.Pin.IN** – Entrada Digital
   - Permite ler sinais HIGH ou LOW
   - Usado para botões, sensores digitais, chaves
3. **machine.Pin.OPEN_DRAIN** – Dreno Aberto
   - Permite conectar múltiplos dispositivos no mesmo barramento
   - Usado em protocolos I2C, comunicação multi-dispositivo

Configurações de Resistor Interno:

- **machine.Pin.PULL_UP**: Resistor pull-up interno (∼50kΩ)
  - Pino fica em HIGH quando desconectado
  - Botão conecta pino ao GND quando pressionado
- **machine.Pin.PULL_DOWN**: Resistor pull-down interno (∼50kΩ)
  - Pino fica em LOW quando desconectado
  - Botão conecta pino ao 3.3V quando pressionado
- **None** (padrão): Sem resistor interno
  - Pino fica "flutuante" – pode ler valores aleatórios
  - Requer resistor externo no circuito

Listing 4: Métodos da classe Pin

```
# Escrita de valores (apenas para pinos de saida)
pino.value(1)       # Define pino como HIGH (3.3V)
pino.value(0)       # Define pino como LOW (0V)
pino.on()           # Atalho para value(1)
pino.off()          # Atalho para value(0)
pino.toggle()       # Inverte estado atual

# Leitura de valores (principalmente para pinos de entrada)
estado = pino.value()  # Retorna 0 ou 1

# Configuracao de interrupcoes (IRQ)
def callback(pino):
    print("Interrupcao detectada!")
```

```
pino.irq(trigger=machine.Pin.IRQ_RISING, handler=callback)
# Triggers disponiveis: IRQ_RISING, IRQ_FALLING, IRQ_RISING | IRQ_FALLING
```

Listing 5: Implementação de debouncing com interrupções

```
import machine
import utime

# Configuracao
botao = machine.Pin(18, machine.Pin.IN, machine.Pin.PULL_DOWN)
led = machine.Pin(16, machine.Pin.OUT)

# Variaveis de controle
ultimo_tempo = 0
DEBOUNCE_TIME = 200  # ms

def botao_pressionado(pino):
    global ultimo_tempo
    tempo_atual = utime.ticks_ms()

    # Ignora pressionamentos muito proximos (debounce)
    if utime.ticks_diff(tempo_atual, ultimo_tempo) > DEBOUNCE_TIME:
        led.toggle()  # Inverte estado do LED
        ultimo_tempo = tempo_atual

# Configura interrupcao
botao.irq(trigger=machine.Pin.IRQ_RISING, handler=botao_pressionado)

# Loop principal vazio - interrupcoes controlam tudo
while True:
    utime.sleep(0.1)
```

Tabela 2: Pinos com funções especiais

Pino GPIO	**Função Alternativa**	**Observações**
GP25	LED Onboard	LED integrado na placa
GP26	ADC0 / A0	Entrada analógica canal 0
GP27	ADC1 / A1	Entrada analógica canal 1
GP28	ADC2 / A2	Entrada analógica canal 2
GP0-GP5	UART0, I2C0, SPI0	Comunicação serial

Pinos Especiais no Raspberry Pi PICO:

Boas Práticas:

1. **Sempre configure resistores pull-up/pull-down em entradas** para evitar leituras flutuantes
2. **Use debouncing** ao trabalhar com botões físicos
3. **Evite curto-circuitos** – nunca configure dois pinos como saída HIGH conectados diretamente
4. **Limite corrente em LEDs** – use resistores apropriados (220Ω-1kΩ)
5. **Documente seus pinos** – mantenha registro de qual pino controla cada componente

1.4 Conclusões

A semana 10 proporcionou compreensão prática e teórica fundamental sobre a diferença entre sinais digitais e analógicos em sistemas embarcados:

1.4.1 Aspectos Teóricos Relevantes:

1. **Sinais Digitais:**
   - Representação binária simplifica processamento computacional
   - Apenas dois estados discretos: HIGH (1) e LOW (0)
   - Imunidade a ruído devido à natureza discreta
   - Processamento direto pelo microcontrolador sem conversão
2. **Sinais Analógicos:**
   - Representam mundo real de forma contínua e precisa
   - Requerem conversão ADC para processamento digital
   - Resolução do ADC determina precisão da medição
   - Essenciais para interfaceamento com sensores físicos
3. **Conversor ADC:**
   - Ponte entre mundo analógico e digital
   - Resolução de 12 bits oferece 4096 níveis distintos
   - Taxa de amostragem afeta qualidade da medição
   - Limitado pela tensão de referência (3.3V no Pico)

1.4.2 Aspectos Práticos Relevantes:

1. **Lição 5 (Digital):**
   - Configuração simples de entrada/saída digital
   - Importância de resistores pull-up/pull-down
   - Resposta instantânea e determinística
   - Ideal para sistemas de controle on/off
2. **Lição 6 (Analógico):**
   - Necessidade de usar portas ADC específicas (A0, A1, A2)
   - Conversão de valores brutos para unidades físicas (V, %, etc.)
   - Implementação de limiares para decisões binárias
   - Capacidade de medir grandezas contínuas

1.4.3 Comparação Prática:

Tabela 3: Comparação entre sinais digitais e analógicos

Aspecto	**Sinal Digital**	**Sinal Analógico**
Complexidade	Baixa	Média
Precisão	Binária (on/off)	Proporcional (0-100%)
Hardware necessário	Pinos GPIO padrão	Pinos ADC específicos
Processamento	Direto	Requer conversão
Aplicações	Botões, chaves, LEDs	Sensores, potenciômetros
Velocidade	Muito rápida	Limitada pelo ADC

1.4.4 Experiência do Grupo:

Todos os integrantes do grupo participaram ativamente das implementações práticas, dividindo as seguintes responsabilidades:

- **Cauã:** Implementação e teste do circuito digital (Lição 5), documentação da classe Pin
- **Murilo:** Implementação e análise do circuito analógico (Lição 6), calibração do ADC
- **Nicolas:** Integração dos códigos, testes comparativos, elaboração do relatório

A experiência foi extremamente enriquecedora, consolidando conceitos teóricos através de aplicações práticas tangíveis. A diferença no tratamento de sinais digitais e analógicos ficou clara através dos experimentos.

Observação

Observação Final:
O domínio desses conceitos é essencial para desenvolvimento de sistemas embarcados complexos. A capacidade de trabalhar tanto com entradas digitais (botões, sensores digitais) quanto analógicas (temperatura, luz, posição) expande enormemente as possibilidades de projetos com o Raspberry Pi PICO.
Possibilidades futuras incluem:

- Sistemas de controle de temperatura com sensor analógico e cooler digital
- Interfaces homem-máquina com botões e displays
- Aquisição de dados de múltiplos sensores analógicos
- Implementação de controladores PID baseados em feedback analógico

1.5 Vídeo de Apresentação

Link para o vídeo: https://drive.google.com/file/d/$1_AqhFjoAb6i7El_KUtWHvguT-7en2qdN/view?usp=sharing$

Referências

1. SEED. *Beginner's Guide to Raspberry Pi PICO*. Disponível no Moodle.
2. Raspberry Pi Foundation. *Getting started with MicroPython on Raspberry Pi Pico*. Disponível em: `https://projects.raspberrypi.org/en/projects/getting-started-with-the-0`
3. MicroPython. *MicroPython Libraries – Official Repository*. Disponível em: `https://docs.micropython.org/en/latest/library/index.html`
4. Material de aula – Slides Semana 10: ArqOrg2-Lab-S10.pdf

2 Relatório Semana 11

2.1 Especificação e Objetivos do Problema

Esta atividade prática teve como objetivo introduzir e aplicar a técnica de **PWM (Pulse Width Modulation)** para gerar sinais analógicos simulados a partir de saídas digitais no microcontrolador **RP2040**, presente no **Raspberry Pi Pico**.

- **Lição 7 – PWM para Controle de LED:**
  - Compreender o conceito de modulação por largura de pulso.
  - Controlar a intensidade luminosa de um LED utilizando PWM.
  - Explorar os parâmetros de frequência e ciclo de trabalho.
- **Lição 8 – PWM para Controle de Campainha (Buzzer):**
  - Gerar sinais de áudio (tons) através do controle de frequência.
  - Utilizar PWM para produzir notas musicais.
  - Compreender a relação entre frequência do sinal e percepção sonora.

2.2 Fundamentação Teórica

PWM – Modulação por Largura de Pulso O PWM é uma técnica que permite controlar a **tensão média** aplicada a uma carga por meio da variação do tempo em que um sinal digital permanece em nível alto (ON) e baixo (OFF). Apesar do microcontrolador operar digitalmente (0V ou 3.3V), é possível simular comportamentos analógicos ajustando o tempo médio de condução elétrica.

- **Ciclo (T):** Tempo total de um pulso completo (ON + OFF).
- **Ciclo de Trabalho (Duty Cycle):** Porcentagem de tempo em que o sinal permanece em nível alto durante um ciclo.
- **Frequência:** Quantidade de ciclos completos por segundo (Hz).

A tensão média fornecida a uma carga pode ser expressa como:

$$V_{mdio} = V_{max} \times \frac{T_{ON}}{T_{total}} = V_{max} \times DutyCycle$$

Quanto maior o **duty cycle**, maior a energia entregue à carga. Isso permite controlar a intensidade de um LED, a velocidade de um motor, ou a altura de um som emitido.

—

2.3 Lição 7: Controle de LED com PWM

Descrição: Nesta lição, o PWM foi utilizado para controlar o brilho de um LED conectado ao pino GP15 do Raspberry Pi Pico. A variação do ciclo de trabalho altera o brilho de forma suave.

Circuito:

- Pino GP15 → LED (via resistor de 220Ω)
- GND → Catodo do LED

Listing 6: Controle de brilho de LED com PWM

```
from machine import Pin, PWM
from time import sleep

# Configuracao do LED no pino 15
led = PWM(Pin(15))
led.freq(1000)  # Frequencia de 1kHz

while True:
    # Aumenta gradualmente o brilho
    for duty in range(0, 65535, 500):
        led.duty_u16(duty)
        sleep(0.001)

    # Diminui gradualmente o brilho
    for duty in range(65535, 0, -500):
        led.duty_u16(duty)
        sleep(0.001)
```

Análise do Código:

- O método `PWM(Pin(15))` cria um sinal PWM no pino GP15.
- `freq(1000)` define a frequência do sinal (1 kHz).
- `duty_u16()` define o ciclo de trabalho com resolução de 16 bits (0–65535).
- O brilho do LED aumenta e diminui gradualmente, simulando um efeito de fade in/out.

Resultados Observados:

✓ LED apresentou variação suave de brilho.

✓ A resposta visual é linear e sem ruídos perceptíveis.

✓ Frequência de 1kHz evita cintilação perceptível ao olho humano.

Conclusão Parcial: O experimento demonstra o uso do PWM para controlar a intensidade de dispositivos analógicos (como LEDs) de forma precisa e eficiente, utilizando apenas sinais digitais.

—

2.4 Lição 8: Controle de Buzzer com PWM

Descrição: Nesta lição, utilizamos o PWM para gerar diferentes frequências sonoras em um buzzer passivo, reproduzindo notas musicais simples. O controle é feito ajustando a frequência do sinal PWM.

Circuito:

- Pino GP16 → Terminal positivo do buzzer
- GND → Terminal negativo

Listing 7: Controle de buzzer com PWM (notas musicais)

```
from machine import Pin, PWM
from time import sleep

# Configuracao do buzzer no pino 16
buzzer = PWM(Pin(16))

# Frequencias das notas (Hz)
NOTAS = {
    "C": 261,  # DO
    "D": 294,  # RE
    "E": 329,  # MI
    "F": 349,  # FA
    "G": 392,  # SOL
    "A": 440,  # LA
    "B": 494   # SI
}

# Tocar escala musical
for nota, freq in NOTAS.items():
    buzzer.freq(freq)
    buzzer.duty_u16(32768)  # 50% de ciclo
    print("Tocando nota:", nota)
    sleep(0.5)
    buzzer.duty_u16(0)  # Desliga som
    sleep(0.1)

buzzer.deinit()
```

Análise do Código:

- A classe `PWM` permite gerar sinais de áudio ajustando a frequência.
- O método `freq()` altera a frequência do sinal PWM, determinando a nota musical.

- O método `duty_u16()` controla o volume (quanto maior, mais intenso o som).
- O método `deinit()` encerra o PWM após o uso.

Resultados Observados:

✓ Cada nota foi reproduzida com frequência correspondente.

✓ O buzzer emitiu som limpo e audível.

✓ Alterações suaves de frequência geram diferentes tons.

Conclusão Parcial: O PWM demonstra versatilidade, sendo aplicável tanto para controle de intensidade (LEDs) quanto para geração de áudio (buzzers), destacando seu papel em sistemas embarcados.

—

2.5 Resumo das Classes da Biblioteca `machine`

Classe `I2C`: Utilizada para comunicação entre o Pico e periféricos compatíveis com o protocolo I2C (como displays, sensores, etc.).

Listing 8: Uso básico da classe I2C

```
from machine import Pin, I2C

# Inicializa I2C no barramento 0 (GP8=SDA, GP9=SCL)
i2c = I2C(0, scl=Pin(9), sda=Pin(8), freq=400000)

# Lista dispositivos conectados
devices = i2c.scan()
print("Dispositivos encontrados:", devices)
```

Métodos principais:

- `I2C(id, scl, sda, freq)` – Inicializa comunicação.
- `scan()` – Retorna lista de endereços detectados.
- `readfrom(addr, nbytes)` – Lê dados de um dispositivo.
- `writeto(addr, buf)` – Envia dados.

Classe `ADC`: Permite leitura de sinais analógicos via conversor A/D interno.

Listing 9: Leitura de sensor analógico com ADC

```
from machine import ADC

sensor = ADC(1)
valor = sensor.read_u16()
tensao = (valor / 65535) * 3.3
print(f"Tens o: {tensao:.2f} V")
```

Classe `PWM`: Gera sinais de modulação por largura de pulso.

Listing 10: Uso básico da classe PWM

```
from machine import Pin, PWM

pwm = PWM(Pin(16))
pwm.freq(1000)
pwm.duty_u16(32768)
```

Principais métodos:

- `freq(f)` – Define frequência (Hz).
- `duty_u16(v)` – Define ciclo de trabalho (0–65535).
- `deinit()` – Desativa o PWM.

—

2.6 Conclusões

- A técnica PWM é essencial em sistemas embarcados para simular sinais analógicos.
- O Raspberry Pi Pico oferece suporte interno a PWM em múltiplos canais.
- A Lição 7 demonstrou controle preciso de brilho (LED).
- A Lição 8 demonstrou controle de som (buzzer) via variação de frequência.
- As classes `I2C`, `ADC` e `PWM` compõem a base de comunicação e controle do hardware.

Observação

Observação Final: O domínio do PWM amplia as possibilidades de aplicações práticas com o Raspberry Pi Pico, permitindo desde controle de potência até a geração de sinais complexos como áudio, motores e dimmers de luz.

2.7 Vídeo de Apresentação

Link para o vídeo: https://drive.google.com/file/d/1ES1vdLbemJPvz-H65kMRWyUUXd4pXlR8/vie

Referências

1. SEED. *Beginner's Guide to Raspberry Pi Pico.*
2. Raspberry Pi Foundation. *Getting Started with MicroPython on Raspberry Pi Pico.*
3. MicroPython Official Docs: `https://docs.micropython.org/en/latest/library/index.html`
4. Material de aula – Slides Semana 11: `ArqOrg2-Lab-S11.pdf`

3 Relatório Semana 12

3.1 Especificação e Objetivos do Problema

Nesta semana o foco foi o desenvolvimento de aplicações com o microcontrolador **RP2040** utilizando um **display LCD 16x2** como interface de saída de informações. A atividade prática está associada à **Lição 9** do material SEED e aos slides da Semana 12 da disciplina, que tratam de:

- Introdução ao funcionamento de displays **LCD** (cristal líquido) e seu uso em sistemas embarcados.
- Utilização do **barramento I²C** para comunicação entre o Raspberry Pi Pico e o módulo **Grove LCD 16x2**.
- Emprego de uma **biblioteca externa** (`lcd1602.py`) para abstrair os detalhes de baixo nível do controlador do LCD.
- Execução da Lição 9, com testes de escrita de mensagens no display, mudança de linhas e simples efeitos de atualização de texto.
- Elaboração de um **resumo sobre as funcionalidades e implementação da biblioteca `lcd1602.py`**.

De acordo com o enunciado da tarefa, o relatório deveria ressaltar tanto os aspectos teóricos (conceito de LCD, I²C, bibliotecas externas) quanto os aspectos práticos da implementação em MicroPython. :contentReference[oaicite:0]index=0

3.2 Metodologia

Hardware

- Raspberry Pi Pico (RP2040).
- Base/Shield compatível com Grove para facilitar a conexão do módulo.
- Módulo **Grove - LCD 16x2** com interface I²C.
- Cabos de conexão Grove ou jumpers (SDA, SCL, VCC, GND).

Software

- IDE **Thonny**, configurada para o Raspberry Pi Pico.
- Firmware **MicroPython** gravado no Pico.
- Biblioteca padrão `machine` para acesso ao barramento I²C.
- Biblioteca de terceiros **`lcd1602.py`**, salva previamente na memória do Pico.

Bibliotecas externas fornecidas O professor disponibilizou o arquivo `libraries.zip`, contendo as bibliotecas `dht11.py`, `dht20.py`, `lcd1602.py` e `ws2812.py`. Na Semana 12, utilizamos especificamente a biblioteca `lcd1602.py`, responsável por encapsular o controle do display LCD 16x2 via barramento I^2C, simplificando o código da aplicação.

Procedimentos

1. **Configuração do ambiente:**
   - Conectar o Pico ao computador via USB.
   - Abrir o Thonny, selecionar a placa *Raspberry Pi Pico* e testar o REPL.
2. **Instalação da biblioteca `lcd1602.py`:**
   - Baixar o arquivo `lcd1602.py` a partir do material disponibilizado.
   - Abrir o arquivo no Thonny e salvá-lo na raiz do sistema de arquivos do Pico com o nome `lcd1602.py`.
3. **Ligação do módulo LCD:**
   - Conectar o conector Grove do LCD na porta correspondente do shield (barramento I^2C).
   - Conferir alimentação (3,3V ou 5V dependendo do módulo) e GND em comum.
4. **Desenvolvimento dos exemplos da Lição 9:**
   - Inicializar o barramento I^2C no Pico.
   - Instanciar o objeto de LCD a partir da biblioteca `lcd1602`.
   - Exibir mensagens em diferentes linhas, testar limpeza de tela e atualização de texto.

3.3 Desenvolvimento e Experimentos Práticos

3.3.1 Fundamentação Teórica: LCD e Cristal Líquido

Displays LCD (*Liquid Crystal Display*) utilizam uma camada de cristal líquido cuja orientação das moléculas é controlada por **campos elétricos** aplicados. Alterando essa orientação, modifica-se o índice de refração do material e, consequentemente, a forma como a luz é polarizada e bloqueada pelos filtros do display.

Alguns pontos importantes:

- O LCD **não emite luz**; ele apenas controla a passagem da luz proveniente de um **backlight** (luz de fundo).
- Um display 16x2 é organizado em uma matriz capaz de exibir **16 caracteres por linha** em **duas linhas**, totalizando 32 caracteres visíveis simultaneamente.
- Cada posição da matriz corresponde a um endereço interno no controlador do LCD, que é acessado via comandos.

3.3.2 Barramento I²C no Raspberry Pi Pico

Para evitar o uso de muitos fios (como em LCDs paralelos tradicionais), o módulo Grove LCD 16x2 utiliza o protocolo **I²C**. Esse barramento possui duas linhas:

- **SDA** – linha de dados seriais.
- **SCL** – linha de clock serial.

No Raspberry Pi Pico:

- O barramento I²C 0 pode ser configurado, por exemplo, com `SDA` no pino `GP8` e `SCL` no pino `GP9`.
- Vários dispositivos podem compartilhar o mesmo barramento, contanto que tenham **endereços I²C** diferentes.

3.3.3 Implementação da Lição 9: Exibição de Mensagens

O primeiro passo no código foi configurar o I²C e criar o objeto do LCD utilizando a biblioteca `lcd1602.py`:

Listing 11: Inicialização do LCD 16x2 via I2C

```
from machine import I2C, Pin
from utime import sleep
from jhd1313m1 import JHD1313M1

# I2C(1) no Pico      SDA=GP6, SCL=GP7
i2c = I2C(1, scl=Pin(7), sda=Pin(6), freq=400000)

lcd = JHD1313M1(i2c)

lcd.clear()
# Rotina de inicializacao do display


# Mensagens iniciais
lcd.print("UFSCar - AOC2", 0, 0)   # linha 0
lcd.print("Semana 12 - LCD", 0, 1)  # linha 1

lcd.set_color(0, 255, 0)   # verde
```

Descrição do fluxo:

- O objeto `i2c` é configurado com frequência de 400 kHz, suficiente para a comunicação com o módulo.
- A função construtora `LCD1602(...)` encapsula o endereço do dispositivo e parâmetros como número de linhas e colunas.

- Após `init()` e `clear()`, o display é preparado para receber texto.
- As funções auxiliares (como `put_line`) foram usadas para escrever diretamente em uma linha específica.

Em seguida, foram realizados testes de atualização dinâmica do conteúdo:

Listing 12: Atualização de texto no LCD 16x2

```
mensagens = [
    "Microcontrolador RP2040",
    "Interface: LCD 16x2",
    "Barramento I2C",
    "Licao 9 - SEED"
]

while True:
    for msg in mensagens:
        lcd.clear()
        lcd.put_line("Exemplo LCD:", 0)
        lcd.put_line(msg[:16], 1)  # garante no max 16 chars
        utime.sleep(2)
```

Resultados Observados:

- ✓ As mensagens foram exibidas corretamente nas duas linhas.
- ✓ A troca cíclica de mensagens tornou claro o funcionamento da limpeza e escrita no display.
- ✓ O uso do I^2C simplificou a conexão física (apenas 4 fios: VCC, GND, SDA, SCL).
- ✓ Pequenos atrasos com `utime.sleep` foram suficientes para garantir estabilidade visual.

3.3.4 Resumo: Biblioteca Externa `lcd1602.py`

A biblioteca `lcd1602.py` foi responsável por encapsular os detalhes de comunicação com o controlador do display. De forma resumida, ela oferece:

Inicialização e Configuração

- Funções e métodos para enviar a sequência de comandos necessária à inicialização do módulo LCD em modo 2 linhas x 16 colunas.
- Configuração do modo de exibição (liga/desliga display, cursor, piscagem, etc.).

Escrita de Caracteres e Linhas Típicas funcionalidades observadas:

- **Posicionamento de cursor**: métodos para posicionar o cursor em uma coluna e linha específicas (por exemplo, linha 0 ou 1 e coluna 0–15).
- **Escrita de caracteres**: rotinas que convertem caracteres ASCII em códigos aceitos pelo controlador do LCD.
- **Escrita de strings**: funções para imprimir uma string completa, avançando automaticamente a posição do cursor.
- **Limpeza**: função `clear()` para limpar todo o conteúdo do display.

Controle de Backlight e Comandos Especiais Dependendo da versão da biblioteca, há ainda:

- Rotinas para ligar e desligar o **backlight** (luz de fundo), útil para economia de energia.
- Funções auxiliares para deslocar o conteúdo (scroll) e para atualizar apenas partes da tela.

Vantagens de usar uma biblioteca externa

1. Evita que o programador precise conhecer em detalhes o **protocolo de comandos** do controlador do LCD (endereços, tempos de espera, etc.).
2. Torna o código principal da aplicação mais limpo e legível, focado na **lógica de negócio** em vez de detalhes de hardware.
3. Facilita a reutilização do código em outros projetos que também usem o mesmo módulo LCD.

3.4 Conclusões

A atividade da Semana 12 consolidou o uso de um **display LCD 16x2** como dispositivo de saída em sistemas embarcados com o Raspberry Pi Pico.

Do ponto de vista teórico, foram reforçados os seguintes conceitos:

- Funcionamento básico de displays LCD baseados em cristal líquido e necessidade de um backlight.
- Organização matricial de caracteres (linhas e colunas) e endereçamento interno do controlador.
- Princípios do barramento **I^2C** (linhas SDA/SCL, endereços de dispositivos, comunicação mestre–escravo).
- Diferença entre bibliotecas padrão e bibliotecas de terceiros em MicroPython, e a importância da reutilização de código.

Em termos práticos, os principais aprendizados foram:

- Configurar corretamente o barramento I^{2}C no Raspberry Pi Pico e testar a presença do módulo via `i2c.scan()`.
- Salvar e utilizar uma biblioteca externa (`lcd1602.py`) no sistema de arquivos do Pico.
- Exibir mensagens em diferentes linhas do LCD, limpar o display e atualizar o conteúdo dinamicamente.
- Perceber, na prática, como uma interface visual dedicada torna o sistema mais independente do computador e mais próximo de um **produto embarcado real**.

3.4.1 Experiência do Grupo

- **Cauã:** Configuração do barramento I^{2}C, conexão física do módulo LCD e testes com `i2c.scan()`.
- **Murilo:** Implementação e testes do código principal da Lição 9, incluindo a rotação de mensagens no display.
- **Nicolas:** Estudo e síntese das funcionalidades da biblioteca `lcd1602.py`, além da redação desta parte do relatório.

A lição deixou evidente a importância de se dominar tanto o **hardware** (conexões, barramentos) quanto o **software** (bibliotecas, abstrações) para construir sistemas embarcados completos.

Observação

Observação Final:
A interface com display LCD será particularmente útil no **Projeto Final da disciplina**, pois permite apresentar ao usuário informações de status, medições e mensagens de debug sem depender do computador. O uso de um barramento serial como o I^{2}C reduz o número de pinos necessários e facilita a integração com outros sensores e módulos no mesmo sistema.

3.5 Vídeo de Apresentação

Link para o vídeo: https://drive.google.com/file/d/1BVF7FzoHQjyFoZ9Hxf$_PdmenVlXqypkS/view?$ *sharing*

Referências Semana 12

1. SEED. *Beginner's Guide to Raspberry Pi Pico* – Lição 9 (LCD 16x2 com I^{2}C).
2. Material de aula – Slides Semana 12: `ArqOrg2-Lab-S12.pdf`.
3. MicroPython Official Docs – Módulo `machine.I2C`: `https://docs.micropython.org/en/latest/library/machine.I2C.html`

4 Relatório Semana 13 – Projeto Final

4.1 Especificação e Objetivos do Projeto

Para o projeto final da disciplina, o grupo escolheu desenvolver o projeto baseado na **Lição 10 – Temperature and Humidity Monitoring** do livro *Beginner's Guide for Raspberry Pi Pico*.

O objetivo principal do projeto é implementar um **monitor inteligente de temperatura e umidade** utilizando o microcontrolador **RP2040** (Raspberry Pi Pico) e os módulos do kit Grove, exibindo as leituras em um **display LCD 16x2** e sinalizando, via **buzzer**, quando os valores medidos saem de uma faixa considerada confortável.

Objetivos específicos:

- Ler, periodicamente, os valores de **temperatura** e **umidade relativa do ar** usando o sensor de temperatura/umidade (Grove).
- Exibir, em tempo real, as leituras no **LCD 16x2** no formato:
  - Linha 1: `Temp: XX C`
  - Linha 2: `Humid: YY %`
- Implementar uma lógica de alarme usando o operador lógico `or`: se a temperatura **ou** a umidade estiverem fora da faixa de conforto, o buzzer deve ser acionado.
- Consolidar o uso combinado de:
  - barramento **I²C** (para o LCD),
  - sensor digital de temperatura/umidade,
  - biblioteca externa `lcd1602.py`,
  - e bibliotecas padrão do MicroPython (`machine`, `utime`).

4.2 Metodologia

Hardware

- Raspberry Pi Pico (RP2040).
- Grove Shield para Raspberry Pi Pico.
- Módulo Grove – **Temperature & Humidity Sensor** (DHT11/DHT20).
- Módulo Grove – **LCD 16x2** com interface I^2C.
- Módulo Grove – **Passive Buzzer**.
- Cabos Grove (dados e alimentação).

Software

- IDE **Thonny** configurada para Raspberry Pi Pico.
- Firmware **MicroPython** gravado no Pico.
- Bibliotecas padrão: `machine`, `utime`.
- Biblioteca externa `lcd1602.py` (fornecida no material da disciplina).
- Biblioteca externa do sensor de umidade/temperatura (por exemplo `dht11.py`).

Procedimentos

1. **Preparação do ambiente:**
   - Conectar o Pico ao computador e abrir o Thonny.
   - Verificar o REPL e o firmware MicroPython.
2. **Instalação das bibliotecas de terceiros:**
   - Abrir os arquivos `lcd1602.py` e `dht11.py` no Thonny.
   - Salvá-los na placa selecionando *Raspberry Pi Pico* como destino.
3. **Montagem do circuito:**
   - Pluggar o Pico no Grove Shield.
   - Conectar o **LCD** à porta **I2C1** do Shield.
   - Conectar o **Temperature & Humidity Sensor** à porta **D18**.
   - Conectar o **buzzer passivo** à porta **D16**.
4. **Desenvolvimento do código:**
   - Configurar o barramento I^2C e criar o objeto do LCD.
   - Criar o objeto do sensor de temperatura/umidade.
   - Ler periodicamente os valores de temperatura e umidade.
   - Exibir as leituras no LCD, utilizando as funções da biblioteca `lcd1602`.
   - Implementar a lógica de alarme com base em limites pré-definidos.

4.3 Desenvolvimento e Experimentos Práticos

4.3.1 Fundamentação Teórica

Sensor de Temperatura e Umidade O sensor de temperatura e umidade (como o DHT11) é um dispositivo digital capaz de medir:

- Temperatura do ambiente (em °C).
- Umidade relativa do ar (em %).

Internamente o módulo realiza a conversão analógico-digital e envia para o microcontrolador apenas os valores já quantizados, através de um protocolo digital simples (utilizado pela biblioteca `dht11.py`).

Faixa de Conforto Térmico Para fins de teste, adotamos uma faixa de conforto arbitrária, por exemplo:

- Temperatura confortável: entre **20°C** e **28°C**;
- Umidade confortável: entre **40%** e **70%**.

Se **(temperatura fora da faixa) OR (umidade fora da faixa)**, o sistema considera que o ambiente está desconfortável e aciona o buzzer.

Uso do Operador Lógico OR A Lição 10 enfatiza o uso do operador lógico `or`. A expressão:

$$(\text{temp_ok} == \text{False}) \text{ or } (\text{hum_ok} == \text{False})$$

será verdadeira sempre que *pelo menos uma* das condições estiver fora do intervalo definido. Essa lógica é diretamente aplicada para decidir se o alarme será disparado.

4.3.2 Código Implementado

A seguir apresentamos o código desenvolvido para o projeto final. Observação: os nomes de funções da biblioteca do sensor podem variar levemente conforme a versão usada no laboratório (por exemplo `dht`, `dht11` ou `dht20`). O fluxo geral, porém, permanece idêntico.

Listing 13: Projeto final – Monitor de Temperatura e Umidade com LCD e Alarme

```
from machine import Pin, I2C
import utime
import dht              # ou dht11.py fornecido no material
from jhd1313m1 import JHD1313M1   # <-- biblioteca do LCD RGB

# ------------------------------
# Configuracao de hardware
# ------------------------------

# I2C1 do Grove Shield (ajustar pinos se necessario)
# Ex.: SDA = GP6, SCL = GP7 (depende do shield)
i2c = I2C(1, sda=Pin(6), scl=Pin(7), freq=400000)

# Objeto do LCD 16x2 (JHD1313M1)
lcd = JHD1313M1(i2c, cols=16, rows=2)
lcd.clear()
lcd.set_color(255, 255, 255)   # backlight branco (opcional)

# Sensor de temperatura e umidade (pino D18 -> GPIO 18)
sensor = dht.DHT11(Pin(18))

# Buzzer passivo (pino D16 -> GPIO 16)
buzzer = Pin(16, Pin.OUT)
```

```
# Limites de conforto
TEMP_MIN = 20
TEMP_MAX = 28
HUM_MIN  = 40
HUM_MAX  = 70

# ----------------------------
# Funcoes auxiliares
# ----------------------------

def tocar_alarme(vezes=3, atraso_ms=150):
    """
    Aciona o buzzer algumas vezes para sinalizar
    que a temperatura ou umidade estao fora da faixa.
    """
    for _ in range(vezes):
        buzzer.value(1)
        utime.sleep_ms(atraso_ms)
        buzzer.value(0)
        utime.sleep_ms(atraso_ms)

def mostrar_leituras(temp, humid):
    """
    Atualiza o LCD com os valores atuais de temperatura e umidade.
    """
    lcd.clear()
    # Linha 0: temperatura
    lcd.print("Temp: {} C".format(temp), col=0, row=0)
    # Linha 1: umidade
    lcd.print("Humid: {} %".format(humid), col=0, row=1)

def atualizar_cor_backlight(temp_ok, hum_ok):
    """
    Muda a cor do backlight do LCD conforme a situacao:
      - Verde: tudo ok
      - Vermelho: algum valor fora da faixa
    """
    if temp_ok and hum_ok:
        lcd.set_color(0, 255, 0)      # verde
    else:
        lcd.set_color(255, 0, 0)      # vermelho

# ----------------------------
# Loop principal
# ----------------------------

print("Monitor de Temp/Umid iniciado.")
```

```
while True:
    try:
        # Leitura do sensor
        sensor.measure()
        temp = sensor.temperature()
        humid = sensor.humidity()

        # Atualiza o LCD
        mostrar_leituras(temp, humid)

        # Verifica se valores estao na faixa de conforto
        temp_ok = (TEMP_MIN <= temp <= TEMP_MAX)
        hum_ok  = (HUM_MIN  <= humid <= HUM_MAX)

        # Atualiza a cor do backlight de acordo com a condicao
        atualizar_cor_backlight(temp_ok, hum_ok)

        # Se temperatura OU umidade estiverem fora da faixa, dispara alarme
        if (not temp_ok) or (not hum_ok):
            tocar_alarme()
        else:
            buzzer.value(0)

        # Intervalo entre leituras
        utime.sleep(2)

    except OSError as e:
        # Em caso de erro de leitura, mostra mensagem no LCD
        lcd.clear()
        lcd.print("Erro no sensor", col=0, row=0)
        lcd.print(str(e)[:16], col=0, row=1)  # limita a 16 chars
        buzzer.value(0)
        # Backlight amarelo para indicar erro (opcional)
        lcd.set_color(255, 255, 0)
        utime.sleep(2)
```

4.3.3 Resultados Observados

Durante os testes:

- O LCD exibiu corretamente os valores de temperatura e umidade atualizados a cada 2 segundos.
- Alterando artificialmente as condições (aquecendo o sensor com a mão, soprando ar úmido etc.), foi possível ultrapassar os limites configurados e acionar o buzzer.
- A lógica usando o operador `or` se mostrou adequada: o alarme era ativado tanto

quando **apenas a temperatura** estava fora da faixa quanto quando **apenas a umidade** se encontrava fora do intervalo.

- O sistema permaneceu estável após longos períodos de execução contínua.

4.4 Conclusões

O projeto final consolidou diversos conceitos trabalhados ao longo das semanas:

- Integração de múltiplos módulos (sensor, LCD e buzzer) em um mesmo sistema embarcado.
- Uso de bibliotecas externas (`lcd1602.py` e `dht11.py/dht.py`) para abstrair detalhes de baixo nível de hardware.
- Aplicação prática de operadores lógicos (em especial o `or`) para implementar regras de decisão simples porém úteis em sistemas de monitoramento.
- Reforço do uso do barramento **I²C** e de GPIOs digitais no Raspberry Pi Pico.

Do ponto de vista de produto, o sistema desenvolvido pode ser visto como um **termohigrômetro inteligente** simples, capaz de monitorar continuamente o ambiente e avisar, por meio de um sinal sonoro, quando as condições deixam de ser confortáveis.

O grupo considera este projeto adequado como conclusão da disciplina, por ser relativamente simples de implementar (comparado às outras lições 11–14), mas ainda assim ilustrar de forma clara a integração de sensores, atuadores, bibliotecas e lógica de controle em sistemas embarcados.

4.5 Vídeo de Apresentação

Link para o vídeo: https://drive.google.com/file/d/1FxG7EUhJABEb9hoR7rjVxOfBPtBgY8T1/view

Referências Semana 13

1. SEED. *Beginner's Guide for Raspberry Pi Pico* – Lesson 10: Temperature and Humidity Monitoring.
2. Material de aula – Slides Projeto Final / Semana 13 (ArqOrg2-Lab-S13.pdf).
3. Documentação do MicroPython – módulos `machine` e `utime`.